SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:	27.10.26	**Duration:**	60 minutes

M. Rudresh 192372339

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	organized, and includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

ANSWER SHEET – SCENARIO-BASED REQUIREMENT ANALYSIS

Project: Smart School Bus Tracking and Student Safety System | **Student:** M. Rudresh | **Reg. No.:** 192372339

1. Study and Analyze the Scenario

Business domain: School transportation management and student safety. The proposed application connects school administrators, drivers, parents and transport staff through a centralized system.

Stakeholders: Students, parents/guardians, drivers, transport coordinators, school administrators and emergency-response staff.

Existing challenges: Parents cannot reliably see live bus locations; manual attendance and boarding records are error-prone; unexpected delays, route deviations and emergencies may be communicated late.

Objectives: Track buses and routes in real time, record student boarding/drop-off, notify parents, detect deviations and delays, maintain incident logs and generate transportation reports.

2. Functional and Non-Functional Requirements

Functional requirements: FR1 Secure role-based login; FR2 manage student/guardian profiles; FR3 manage buses and drivers; FR4 create and assign routes/stops; FR5 receive and display GPS locations; FR6 record boarding and drop-off with timestamps; FR7 show ETA and bus status; FR8 notify parents of arrivals, delays and boarding events; FR9 detect route deviation; FR10 log incidents and generate reports.

Non-functional requirements: Performance: display recent GPS updates within 10 seconds under normal network conditions. Security: HTTPS, role-based access, encrypted sensitive records and audit logs. Reliability: queue updates during connection loss and synchronize later. Usability: simple mobile-friendly screens. Scalability: support multiple buses and schools. Maintainability: modular services and documented APIs. Availability: target 99.5% monthly uptime (excluding planned maintenance).

3. Actors and Use Cases

Primary actors: Parent (view location/ETA, receive alerts); Driver (start trip, update status, report incident); Administrator/Transport Coordinator (manage students, buses, routes and reports). **Secondary actors:** GPS device/location service (send positions), notification gateway (deliver messages).

Use case diagram: The following is a simplified UML-style system boundary and actor interaction diagram.

Actor	Interaction with School Bus Safety System
Parent	View live bus/ETA; receive boarding, drop-off and delay alerts
Driver	Start/end trip; confirm boarding/drop-off; report incident
Administrator	Manage profiles, buses, drivers and routes; view reports
GPS service	Provide location updates to tracking use case
Notification service	Send event and emergency notifications

4. Software Requirement Specification (SRS)

Introduction: This SRS defines a web/mobile school bus monitoring platform designed to improve safe transportation and parent visibility.

Problem description: Existing manual and fragmented processes cannot provide timely bus positions, verified boarding events or prompt incident communication.

Scope: Manage student and vehicle records, scheduled routes, GPS-based trip tracking, event notifications, incident reporting and dashboards. It does not control vehicle driving or replace emergency services.

Functional requirements: FR1–FR10 above. **Non-functional requirements:** Security, latency, reliability, usability, scalability, maintainability and availability targets defined above.

Use case description – Track Bus: Actor: Parent. Precondition: authorized account and active trip. Trigger: opens tracking screen. Main flow: system verifies permission, fetches latest bus position and displays route/ETA. Alternate flow: when GPS is unavailable, display last update time and an unavailable status. Postcondition: parent sees permitted trip information.

Use case description – Record Boarding: Actor: Driver/authorized staff. Precondition: active trip and student assigned to bus. Flow: identify student, record event/time/location, save entry and send guardian notification. Exception: offline events are queued and synchronized when network returns.

Data requirements: Student ID, guardian contact, bus/driver IDs, assigned route and stops, trip schedule, GPS coordinates/timestamps, boarding events, alerts, incidents and audit records. Apply retention and access rules to student data.

External interfaces: Parent mobile/web UI, driver trip UI, admin dashboard, GPS/location API, push/SMS notification gateway, secure database and HTTPS REST API.

System features: Real-time map, trip history, ETA display, geofence-based deviation alert, student boarding verification, emergency alert workflow and downloadable reports.

5. Assumptions and Constraints

Assumptions: Each bus has a functioning GPS-enabled device; parents have registered contacts; school provides correct routes and rosters; staff are trained; mobile connectivity is usually available.

Constraints: Technical—GPS error and intermittent networks; operational—staff must confirm student events; economic—device, data and messaging costs; legal—child privacy, consent and data protection requirements; time—limited development/testing schedule.

6. Validate Requirement Completeness and Consistency

Completeness: Trace parents to tracking/alerts (FR7–FR8), drivers to trip/event logging (FR6, FR10), administrators to management/reporting (FR2–FR4, FR10), and GPS to location tracking (FR5, FR9).

Consistency and ambiguity: Define “real-time” as target 10-second display latency; define deviation as exceeding a configured route corridor for a configured duration; use a single student-trip-event record to prevent duplicate boarding records.

Improvements and validation: Review requirements with parents, drivers and administrators; test role permissions, offline synchronization, false GPS alarms, notification failures and emergency escalation. Use acceptance tests and a requirement traceability matrix before approval.